

14. Bölüm

İstisna Yönetimi

14.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda bir C program örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
    if (sayi<0) return -1; //Aslında sıfırdan küçük sayının faktöriyeli -1 değil, tanımsızdır.
    if(sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
}
int main() {
    int okunan,deger,hata;
    do {
        printf("Bir Sayı Giriniz:");
        scanf("%d",&okunan);
        deger=faktoriyel(okunan);
        printf("%d nin faktöriyeli: %d\n", okunan, deger);
        if (deger==-1) {
            hata=100;
            break; // exit(100);
        }
    } while (okunan!=0);
    return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu program örneğinde `faktoriyel` fonksiyonu hata durumunda `-1` geri döndürmektedir. Ancak gerçek hayatta bu fonksiyonun böyle bir değer döndürmesi mümkün değildir.

14.2 İstisna Yönetimi

Nesne Yönelimli Programlamada ise **istisna** (**exception**), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, çalışma sırasında (**run-time**) oluşur. Bu tür beklenmeyen durumlarda imal edilen istisna nesneleri (**exception object**) vardır. İstisna iki türlü ortaya çıkar;

1. **Eşzamanlı** (**synchronous**): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
2. **Zaman Uyumsuz** (**asynchronous**): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar kelimeleri kullanılır. İstisnaları işlemek için sözde kod aşağıda verilmiştir;

```
try {
    /*
    ...
    İstisnai Durumun Ortaya Çıkabileceği Kod...
    ...
    */
```

```
throw SomeExceptionType("İstisnai Durum Açıklaması");
/* throw an exception: istisnai bir duruma düşme */
} catch( ExceptionName1 e1 ) {
/* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır. */
}
```

Bu sözde kod şu şekilde açıklanabilir;

- ◊ **try talimatı:** İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla **catch** bloğu takip eder.
- ◊ **catch talimatı:** **try** bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnayı işlemek için kullanılan kod, **catch** bloğunun içine yazılır.
- ◊ **İstisnai duruma düşme:** Bir istisnai duruma **throw** anahtar sözcüğü kullanılarak da düşülebilir. Bu aynı zamanda istisna nesneleri havuzuna **bir istisna fırlatmadır (throw an exception)**.
- ◊ **İstisna sonrası süreç:** Bir program bir **throw** ifadesiyle karşılaştığında hemen içinde bulunduğu **try** bloğu dışına çıkılır ve ilgili istisnayı işlemek için eşleşen bir **catch** bloğu bulmaya başlar.
- ◊ Bir **try** bloğu içinde birden fazla **throw** ifadesi bulunabilir.

Bu yeni bakış açısıyla 14.1 başlığındaki yapısal programlama faktöriyel örneği aşağıdaki gibi yazabiliriz;

```
#include <iostream>
// Faktöriyel fonksiyonu: Hata durumunda istisna havuzuna istisna nesnesi fırlatır
int faktoriyel(int sayi) {
    if (sayi < 0) {
        throw ("Negatif sayilarin faktoriyeli hesaplanamaz!");
        // Metin olarak hata nesnesi oluşturduk.
    }
    if (sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
}
int main() {
    int okunan;

    do {
        std::cout << "Bir Sayi Giriniz (Cikis icin 0): ";
        std::cin >> okunan;
        if (okunan == 0) break;
        try { // Hata riski olan kodu try bloğu içine alıyoruz
            int sonuc = faktoriyel(okunan);
            std::cout << okunan << " sayisinin faktoriyeli: " << sonuc << std::endl;
        }
        catch (const char* e) { // Metin olarak imal edilmiş hata nesnesini yakalıyoruz.
            std::cerr << "HATA: " << e << std::endl;
            return -1;
        }

    } while (okunan != 0);
}
/*Program Çıktısı:
Bir Sayi Giriniz (Cikis icin 0): -5
HATA: Negatif sayilarin faktoriyeli hesaplanamaz!
*/
```

C++ dilinde istisna işleme yapısal programdakinden farklıdır. Şöyle ki;

- ◊ Hata işleme kodu, fonksiyonel koddan yani işin gereği yazılan koddan ayrılır.
- ◊ Fonksiyonlar ve yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir fonksiyon yada yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını **catch** bloğunda işlemeyi seçebiliriz. Geri kalanlarını yöntemi çağıran yere bırakabiliriz. Böylece istisnayı kimin işleyeceğini programcı seçebilir.
- ◊ Hem temel tiplerden (**int**, **float**, **char**,...) hem de nesneler (**std::string**,...) istisna nesnesi imal edilebilir. Böylece hata türlerinin gruplanması yapılabilir.

14.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan **runtime_error** istisnası imal edilmiştir. Bu tip istisna tipleri için projemize **<stdexcept>** başlığı dahil

edilmelidir.

```
#include <iostream>
#include <stdexcept> // runtime_error için gerekli
int main() {
    std::cout << "Burasi her zaman icra edilir." << std::endl;
    try {
        // try BLOĞU: İstisna oluşma ihtimali olan kod bloğu
        int bolunen = 10;
        int bolen = 0;
        int sonuc;
        if (bolen == 0) {
            // std:: ön eki eklendi
            throw std::runtime_error("Sifira Bolmeye Izin Verilmez!");

            // throw satırından sonraki kodlar asla çalışmaz
            std::cout << "Burasi hicbir zaman icra edilmez." << std::endl;
        }
        sonuc = bolunen / bolen;
        std::cout << "Bolme Sonucu: " << sonuc << std::endl;
    } catch (const std::exception& e) {
        // catch BLOĞU: Hata yakalandığında buraya atlanır
        std::cout << "Istisna Yakalandi: " << e.what() << std::endl;
    }
    std::cout << "Burasi da her zaman icra edilir." << std::endl;
}
/*Program Çıktısı:
Burasi her zaman icra edilir.
Istisna Yakalandi: Sifira Bolmeye Izin Verilmez!
Burasi da her zaman icra edilir.
*/
```

Yukarıdaki örnekte `try` bloğunda izin verilmeyen bölme işleminde istisnai bir durum fark ediliyor ve istisna nesnesi imal ediliyor ve `throw` ile fırlatılıyor (**throw an exception**). `catch` bloğunda ise (istisna nesneleri havuzundan) **istisna nesnesi yakalanarak** (**catch an exception**) işleniyor. Örnekte standart tipte istisna nesnesi kullanılmıştır. Bu tiplerin hepsi `std::exception` sınıfından türetilmiştir.

İstisna Sınıfı	Üst Kategori	Açıklama
<code>std::logic_error</code>	<code>std::exception</code>	Programın mantıksal kurgusundaki hatalar (derleme öncesi teorik olarak önlenabilir).
<code>std::invalid_argument</code>	<code>logic_error</code>	Fonksiyona gönderilen argümanın geçersiz olması.
<code>std::domain_error</code>	<code>logic_error</code>	Matematiksel bir işlemin tanım kümesi dışında kalması (örn. negatif sayının karekökü).
<code>std::length_error</code>	<code>logic_error</code>	Nesne boyutunun (örn. <code>std::string</code>) izin verilen maksimum sınırı aşması.
<code>std::out_of_range</code>	<code>logic_error</code>	İndis veya anahtarın geçerli aralığın dışında olması (<code>at()</code> metodu gibi).
<code>std::runtime_error</code>	<code>std::exception</code>	Sadece program çalışırken tespit edilebilen, tahmin edilmesi zor hatalar.
<code>std::range_error</code>	<code>runtime_error</code>	Hesaplama sonucunun içsel bir aralığın dışına çıkması (aritmetik olmayan durumlar).
<code>std::overflow_error</code>	<code>runtime_error</code>	Aritmetik bir işlemin sonucunun veri tipi sınırlarını aşması (üstten taşma).
<code>std::underflow_error</code>	<code>runtime_error</code>	Çok küçük bir sonucun veri tipi tarafından temsil edilememesi (alttan taşma).
<code>std::bad_alloc</code>	<code>std::exception</code>	<code>new</code> operatörü ile bellek tahsisi yapılamaması (bellek yetersizliği).
<code>std::bad_cast</code>	<code>std::exception</code>	<code>dynamic_cast</code> ile yapılan referans dönüşümünün başarısız olması.

Tablo 14.1: Standart İstisna Hiyerarşisi ve Kullanım Alanları

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir. Ama tüm `catch` talimatlarıyla bu istisnalar yakalanmayabilir. Ayrıca tüm istisna nesnelerin tipleri de bilinmeyebilir. Bu durumda varsayılan

`catch` kullanılır Aşağıdaki buna ilişkin örnek verilmiştir..

```

1  #include <iostream>
2  int main() {
3      try {
4          // DİKKAT: throw ile ilk istisna nesnesi oluştuğunda program hemen uygun 'catch' bloğuna
           ↳ atlar.
5          // Alttaki throw satırlarına asla ulaşamaz.
6
7          throw 10; /* Bu satır çalışır ve 13. satırdaki 'int' catch bloğuna gider. */
8
9          // Aşağıdaki satırlar ölü koddur (unreachable code):
10         throw "Hata";
11         throw 'A';
12     }
13     catch (int excpInt) {
14         std::cout << "Tam Sayı Istisnasi Yakalandı: " << excpInt << std::endl;
15     }
16     catch (const char* excpStr) {
17         // Modern C++'ta string literalleri 'const char*' olarak yakalanmalıdır.
18         std::cout << "Metin Istisnasi Yakalandı: " << excpStr << std::endl;
19     }
20     catch (char excpChar) {
21         std::cout << "Karakter Istisnasi Yakalandı: " << excpChar << std::endl;
22     }
23     catch (...) {
24         // Default catch: Yukarıdakilere uymayan her şey buraya düşer.
25         std::cout << "Diğer Catchlerde islenmeyen bir istisna burada islendi." << std::endl;
26     }
27 }
28 /* Programın Çıktısı:
29 Tam Sayı Istisnasi Yakalandı: 10
30 */

```

Aşağıda `Kisi` sınıfta `yas` özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

#include <iostream>
#include <stdexcept>
#include <string>
class Kisi {
private:
    int yas;
public:
    Kisi() : yas(15) {} // Yapıcı (Constructor)
    int getYas() const { // Nesneyi değiştirmedığı için 'const' eklemek iyi bir pratiktir
        return yas;
    }
    void setYas(int pYas) {
        if (pYas < 0)
            throw std::range_error("Gecersiz Yas Degeri: <0!");
        else if (pYas > 120)
            throw std::range_error("Cok Buyuk Yas Degeri: >120!");
        yas = pYas;
    }
};

int main() {
    Kisi ali;
    try {
        // 1. Geçerli bir değer atama
        ali.setYas(12);
        std::cout << "Guncel Yas: " << ali.getYas() << std::endl;
        // 2. İstisna yaratacak değer (Hata fırlatıldığı an try bloğu terk edilir)
        ali.setYas(-1);
        // --- BU SATIRDAN SONRASI İCRA EDİLMEZ ---
    }
}

```

```

std::cout << "Bu satira asla ulasilamayacak: " << ali.getYas() << std::endl;
ali.setYas(130);
std::cout << ali.getYas() << std::endl;
}
catch (const std::exception& istisna) {
    // Yakalanan istisna nesnesinin içeriğini yazdırıyoruz
    std::cout << "İstisnai Bir Durum Yakalandi!" << std::endl;
    std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
}
}
/*Program Çıktısı:
Guncel Yas: 12
İstisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/

```

14.4 std::exception ve what() Yöntemi

C++'ta tüm standart hata sınıfları (örneğin `std::out_of_range`, `std::bad_alloc`) `std::exception` sınıfından türetilmiştir. Bu sınıfın içinde `what()` adında sanal bir yöntem bulunur. Bu yöntemin görevi, **hatayı açıklayan bir karakter dizisi** (`const char*`) döndürmektir. Bu yöntem `catch` bloğu içinde hata nesnesi üzerinden çağrılır.

```

#include <iostream>
#include <vector>
#include <stdexcept> // Standart istisnalar için
int main() {
    std::vector<int> sayilar = {10, 20, 30};
    try {
        // Vektörün sınırları dışında bir elemana erişmeye çalışıyoruz (indis 5 yok)
        // .at() fonksiyonu hata oluşunca std::out_of_range fırlatır.
        std::cout << sayilar.at(5) << std::endl;
    }
    catch (const std::exception& e) {
        // 'e' nesnesi üzerinden what() çağrılır
        std::cout << "Hata Yakalandi: " << e.what() << std::endl;
    }
}
/*
Program Çıktısı:
Hata Yakalandi: vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
*/

```

`std::exception` ve `what()` kullanmalıyız! Çünkü;

- ♦ **Hiyerarşi:** Tüm hataları (`const std::exception& e`) referansı ile tek bir `catch` bloğunda yakalayabilirsiniz. Arka planda çok biçimlilik çalışır.
- ♦ **Bilgi Paylaşımı:** Hatanın nedenini kodun her yerinde manuel `cout` yapmak yerine, hata nesnesinin içinde taşırsınız.
- ♦ **Standartlara Uyumluluk:** Büyük kütüphaneler (ileride anlatacağımız STL gibi) bu yapıyı kullanır. Sizin kodunuzun da bu yapıya uyması, kodun profesyonelliğini artırır.

catch ile İstisna Yakalama

İyi bir istisna yönetimi için `catch (std::exception e)` yerine `catch (const std::exception& e)` kullanın! Bu, nesnenin kopyalanmasını önler ve **nesne dilimleme** (`object slicing`) problemini engeller. Ayrıca her zaman önce en özel hata sınıflarını yakalanmalı ve en son ise genel `std::exception` sınıfını yakalanmalıdır.

14.5 noexcept Anahtar Kelimesi

C++'ta `noexcept` anahtar kelimesi, hem bir sıfat/belirleyici (`noexcept specifier`) hem de bir işleç (`noexcept operator`) olarak görev yapan, C++11 ve sonrası performans optimizasyonunun temel taşlarından biridir.

§14.5.1 noexcept İşleci

noexcept işleci (**noexcept operator**), bir ifadenin (**expression**) istisna (**exception**) fırlatıp fırlatmayacağını **derleme zamanında** (**compile-time**) kontrol eder. Sonuç olarak **bool** bir değer döndürür:

1. **true**: Eğer ifade istisna fırlatmayacağı garanti edilmişse.
2. **false**: Eğer ifade istisna fırlatabilirse.

Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
#include <stdexcept>
void fonksiyon() { throw std::runtime_error("oops"); }
void bosfonksiyon() {} // Belirteç kullanılmadığı için potansiyel tehlike!
struct yapi {};
int main() {
    // 1. fonksiyon() içinde throw var, derleyici bunun fırlatabileceğini biliyor.
    std::cout << noexcept(fonksiyon()) << std::endl;    // Çıktı: 0
    // 2. bosfonksiyon() boş olsa bile 'noexcept' olarak işaretlenmediği için
    // derleyici her ihtimale karşı istisna fırlatabilir kabul eder.
    std::cout << noexcept(bosfonksiyon()) << std::endl; // Çıktı: 0
    // 3. Temel aritmetik işlemler istisna fırlatmaz.
    std::cout << noexcept(1 + 1) << std::endl;          // Çıktı: 1
    // 4. Basit bir struct'ın varsayılan yapıcısı (constructor) istisna fırlatmaz.
    std::cout << noexcept(yapi()) << std::endl;         // Çıktı: 1
}
```

§14.5.2 noexcept Belirleyicisi

noexcept Belirleyicisi (**noexcept specifier**), bir fonksiyonun istisna fırlatmayacağını derleyiciye söz vererek bildirmek için fonksiyon imzasında kullanılır. Eğer bu sözü verirseniz, **noexcept** işleci bu fonksiyon için **true** döndürür.

```
#include <iostream>
void guvenliFonksiyon() noexcept {
    // Bu fonksiyonun hata fırlatmayacağı garanti edildi.
}
int main() {
    std::cout << noexcept(guvenliFonksiyon()); // Çıktı: 1
}
```

14.6 Kullanıcı Tanımlı İstisna Tipleri

Her zaman standart istisna nesneleri kullanmak zorunda değiliz. Bu durumda **<exception>** başlığını projemize dahil ederek kendi istisna nesnemizi de tanımlayabiliriz. Aşağıdaki örnekte profesyonel yaklaşımdan biri sergilenmiştir. **std::exception** sınıfından kendi sınıfını türetilmiş ve **what()** fonksiyonunu kendi hata mesajı geçersiz kılınmaktadır (**override**).

```
#include <iostream>
#include <exception>
#include <string>
// Özel hata sınıfı: Maksimum yaş sınırı için
struct yasMaxException : public std::exception {
    // C++11 ve sonrasında 'throw()' yerine 'noexcept' kullanılır
    // what() fonksiyonunu geçersiz kılıyoruz (override)
    // 'noexcept' bu fonksiyonun kendisinin hata fırlatmayacağını garanti eder
    const char* what() const noexcept override {
        return "Cok Buyuk Yas Degeri: >120!";
    }
};
// Özel hata sınıfı: Minimum yaş sınırı için
struct yasMinException : public std::exception {
    // what() fonksiyonunu geçersiz kılıyoruz (override)
    // 'noexcept' bu fonksiyonun kendisinin hata fırlatmayacağını garanti eder
    const char* what() const noexcept override {
        return "Gecersiz Yas Degeri: <0!";
    }
}
```

```

};
class Kisi {
private:
    int yas;
public:
    Kisi() : yas(15) {}
    int getYas() const { return yas; }
    void setYas(int pYas) {
        if (pYas < 0)
            throw yasMinException(); // Özel hata nesnesi fırlatılıyor
        else if (pYas > 120)
            throw yasMaxException();
        yas = pYas;
    }
};

int main() {
    Kisi ali;
    try {
        ali.setYas(12);
        std::cout << ali.getYas() << std::endl;
        // Hata burada fırlatılacak
        ali.setYas(-1);
        // Bu satırlar icra edilmez
        std::cout << ali.getYas() << std::endl;
        ali.setYas(130);
    }
    catch (const std::exception& istisna) {
        // Polimorfizm sayesinde her iki özel hata da burada yakalanır
        std::cout << "İstisnai Bir Durum Yakalandı!" << std::endl;
        std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
    }
}

/*Program Çıktısı:
12
İstisnai Bir Durum Yakalandı!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/

```

14.7 Yığın Çözülmesi

Yığın Çözülmesi (**stack unwinding**), C++'ın bellek yönetimi ve hata güvenliği (**exception safety**) konusundaki en "kahraman" özelliklerinden biridir. İstisnalar fırlatıldığında arka planda gerçekleşen bu süreci anlamak, profesyonel bir C++ geliştiricisi olmak için şarttır.

Bir fonksiyon içinde **throw** ile hata fırlatıldığında, çalışma zamanında (**run-time**) bu hatayı yakalayacak uygun bir **catch** bloğu aramaya başlar. Bu arama sırasında, hatanın fırlatıldığı noktadan **catch** bloğuna kadar olan tüm yerel nesnelerin yıkıcıları (**destructor**) otomatik olarak çağrılır. İşte bu temizlik sürecine **yığın çözülmesi** (**stack unwinding**) denir.

Süreç şöyle işler;

1. **Hata Fırlatılır:** Bir fonksiyonda **throw** satırı çalışır.
2. **Geriye Doğru Tarama:** Program o anki fonksiyondan çıkar ve çağırıcı (**caller**) fonksiyona geri döner.
3. **Otomatik Temizlik:** Çıkan her fonksiyondaki yerel (stack üzerindeki) nesnelerin yıkıcıları (**~ClassName**) sistem tarafından sırayla çağrılır.
4. **Catch Bloğu Bulunur:** Uygun bir **catch** bloğuna ulaşılan kadar bu süreç "yukarı doğru" devam eder. Eğer **main()** fonksiyonuna kadar uygun bir blok bulunamazsa, **std::terminate** çağrılır ve program çöker.

Bunu aşağıdaki örneği inceleyerek anlayabiliriz;

```

#include <iostream>
#include <string>
class Kaynak {
    std::string ad;

```



```

public:
Kaynak(std::string n) : ad(n) { std::cout << ad << " acildi.\n"; }
~Kaynak() { std::cout << ad << " otomatik olarak KAPATILDI (Yikici calisti).\n"; }
};

void fonksiyonB() {
    Kaynak b_kaynagi("FonksiyonB Kaynagi");
    std::cout << "FonksiyonB icinde hata firlatiliyor...\n";
    throw std::runtime_error("Kritik Hata!"); // Hata burada firlatilir
    std::cout << "Bu satir asla calismaz.\n";
}

void fonksiyonA() {
    Kaynak a_kaynagi("FonksiyonA Kaynagi");
    fonksiyonB();
    std::cout << "FonksiyonA devam ediyor...\n"; // Buraya asla gelinmez
}

int main() {
    try {
        fonksiyonA();
    }
    catch (const std::exception& e) {
        std::cout << "\nCatch Blogu Yakaladi: " << e.what() << "\n\n";
    }
    return 0;
}

/*Program Çıktısı:
FonksiyonA Kaynagi acildi.
FonksiyonB Kaynagi acildi.
FonksiyonB icinde hata firlatiliyor...
FonksiyonB Kaynagi otomatik olarak KAPATILDI (Yikici calisti).
FonksiyonA Kaynagi otomatik olarak KAPATILDI (Yikici calisti).

Catch Blogu Yakaladi: Kritik Hata!
*/

```

Yığın çözülmesi, C++'taki RAIİ (**Resource Acquisition Is Initialization**) prensibinin temelidir ve 15.7.1 başlığında incelenecektir. Eğer bu özellik olmasaydı:

- ◊ Açık kalan dosyalar kapanmazdı.
- ◊ Kilitlenmiş (**mutex**) veriler kilitli kalırdı (**deadlock**).
- ◊ Bellek sızıntıları (**memory leaks**) kaçınılmaz olurdu.

Yıkıcılar Hata Fırlatmamalı

Yığın çözülmesi sırasında bir nesnenin yıkıcısı (**~ClassName**) çalışırken, o yıkıcının içinden **yeni bir hata fırlatılmamalıdır!** Eğer bir istisna işlenirken (yığın çözülmesi sürerken) ikinci bir istisna fırlatılırsa, C++ çalışma zamanı ne yapacağını bilemez ve programı anında sonlandırır (**std::terminate**). Bu nedenle yıkıcılar her zaman güvenli olmalı ve hataları kendi içinde yutmalıdır (**noexcept**).

İstisnalar sadece bir hata bildirme yöntemi değildir; aynı zamanda C++'ın yığın belleğindeki nesneleri güvenli bir şekilde imha etme garantisidir. **throw** ile **catch** arasındaki mesafe ne kadar uzak olursa olsun, aradaki tüm yerel nesneler usulünce yok edilir.

14.8 Düşün ve Çöz

- ◊ **Düşün:** Geleneksel hata kodu döndürme yöntemi ile C++ istisna mekanizması arasındaki farkı karşılaştırın. İstisnalar her durumda daha iyi bir çözüm müdür? Gömülü sistemlerde istisnalar neden sorunlu olabilir?
- ◊ **Çöz:** **try-catch-throw** mekanizmasını kullanarak bir bölme işleminin sıfıra bölme hatasını zarif biçimde ele alan bir program yazın. Farklı istisna tiplerini **std::exception** ve **std::runtime_error** birbiriyle karşılaştırın.
- ◊ **Düşün:** İstisna güvenliği (**exception safety**) kavramını açıklayın. Temel garanti, güçlü garanti ve hata atmama garantisi (**nothrow guarantee**) arasındaki farklar nelerdir? Her biri için örnek verin.
- ◊ **Çöz:** Özel bir istisna sınıfı (**class DivisionByZeroException**) tasarlayın. Bu sınıfı **std::exception**'dan

- türeterek `what()` yöntemini geçersiz kılın (**override**) ve kullanımını bir örnekle gösterin.
- ◇ **Düşün:** `noexcept` sıfatının anlamı nedir? Derleyici optimizasyonları açısından `noexcept` neden önemlidir? Hangi tür fonksiyonlar `noexcept` olarak işaretlenmelidir?